

Matrix Inversion on Hardware Board Using FPGA for Application in MIMO Systems

Ranjith Reddy Lingala
DESE, Microelectronics and VLSI
Indian Institute of Sciences
Bangalore, India
ranjithreddy@iisc.ac.in

Jogesh Chandra Dash
Electrical Communication Engineering
Indian Institute of Sciences
Bangalore, India
jogeshdash@iisc.ac.in

Debdeep Sarkar
Electrical Communication Engineering
Indian Institute of Sciences
Bangalore, India
debdeep@iisc.ac.in

Abstract—This paper presents a new method for obtaining the matrix inversion using FPGA. In this method the characteristics of a 2×2 matrix inversion and block matrix inversion are used in an iterative manner to obtain the matrix inversion. The performance of this algorithm is satisfactory at the low computation load compared to the traditional row/column wise methods. Here, the comparison of various decomposition methods using MATLAB results is done and because of some notable advantages over other methods, LDL^T decomposition is used in the proposed inversion algorithm. The proposed method can be implemented in MIMO systems which can add an advantage in terms of processing time and better accuracy.

Index Terms—QR, LU, CHOLESKY, LDL^T , FPGA (Field Programmable Gate Array), MIMO (Multi-Input Multi-Output).

I. INTRODUCTION

Matrices are employed in various applications such as MIMO systems, radar signal processing, control systems, video processing, image processing, compressive sensing and many more. Multiple-Input Multiple-Output (MIMO) technology is an essential part in communication systems [1], [2], [3]. It has contributed to the enhancement of high-rate wireless communications. MIMO technology is a promising technology that will contribute to (5G) communication system. Consider a MIMO wireless system with M number of transmitting antennas and N number of receiving antennas to explain the importance of matrix inversion in hardware. Acquiring the characteristics of channel matrix (H) from the training symbols, the transmitted signals (X) can be obtained using the received signals (Y) at the receiver, by multiplying the inverse channel matrix (H^{-1}) and received vector ($Y=HX$, neglecting noise) as shown in (1).

$$X = H^{-1} \times Y \quad (1)$$

H^{-1} can be obtained in hardware and software. Hardware implementation of matrix inversion requires lower power consumption, lower time complexity, and area optimization. Particularly, FPGA, a reprogrammable integrated circuit, driven by their flexibility, increased system performance, total power

reduction, hardware-timed speed and reliability, and parallelism. Precisely, an application developed in FPGA hardware is for performance acceleration. Whereas software can't achieve satisfactory speed compared to hardware. In view of above advantages and due to the high energy efficiency ratio, the implementation of the matrix inversion is focused on FPGA.

In this work, we propose an algorithm based on LDL^T decomposition which shows better accuracy and performance with less resource usage and faster computation time in hardware (FPGA).

This paper is organized as follows. Section II presents various factorization techniques. Section III presents proposed matrix inversion method. Section IV presents performance comparison of different matrix decomposition methods. Section V discusses the implementation of LDL^T decomposition in hardware and its architecture. Section VI presents conclusion.

II. FACTORIZATION TECHNIQUES

Any matrix inversion problem to solve it in analytic way or to make it computationally simple, one should convert it into an easy decomposition problem. Various factorization techniques discussed here are CHOLESKY, LDL^T , LU, QR [3], [4].

- Cholesky decomposition factorizes into a lower triangular matrix (L) and its conjugate transpose (complex matrix) or its transpose (real matrix) (L^*).
- LDL^T decomposition factorizes into a lower triangular matrix (L), a diagonal matrix (D) and conjugate transpose of the lower triangular matrix (L^T) [5].
- LU Decomposition factorizes a positive definite matrix in the form of lower triangular matrix (L) with diagonal elements "1" and upper triangular matrix (U).
- QR Decomposition factorizes into a product ($Q \times R$) of an orthogonal matrix (Q) and an upper triangular matrix (R).

III. MATRIX INVERSION ALGORITHMS

There are various matrix inversion algorithms available in literature, some of those algorithms are Upper Triangular matrix inversion [6], Diagonal-wise algorithm [5], Recursive Diagonal-wise algorithm [5], SPMI (Simple Positive definite

symmetric Matrix Inversion) algorithm [7], Recursive algorithm for block sub-matrices [10].

The Diagonal-wise algorithm proposed in [5] is based on the algorithm proposed by the author of [9] to avoid computing the intermediate results. Diagonal-wise algorithm makes use of characteristics of the upper triangular matrix of size $N \times N$, and for $(N+k) \times (N+k)$ matrix size an algorithm is proposed, they use block matrix inversion method as shown in equ. (6), (7) to calculate the extra K part of the matrix size.

A. Proposed Extended Recursive Diagonal-wise algorithm.

Recursive Diagonal-wise algorithm proposed in [5] uses characteristics of $N \times N$ matrix size, this consumes big amount of hardware resource. An extension to the existing algorithm is using the characteristic of only 2×2 inverse lower triangular matrix and iterating from 3 to N (matrix size) using block matrix inversion to obtain matrix inversion of dimension $N \times N$. Initially any matrix (M) is converted to symmetric matrix ($A = M \times M^T$) to apply LDL^T factorization (2). Further, Recursive diagonal wise algorithm is applied after LDL^T factorization. The lower triangular matrix (3) and diagonal matrix (4) will be as shown below. The lower triangular matrix (3) contains diagonal elements as “1”, this adds as an advantage in the computation of inversion. The inversion of the matrix A will be of the form as shown (5).

$$A = LDL^T \quad (2)$$

$$L_{N \times N} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ l_1 & 1 & 0 & \dots & 0 \\ l_2 & l_3 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ l_4 & l_5 & l_6 & \dots & 1 \end{bmatrix} \quad (3)$$

$$D_{N \times N} = \begin{bmatrix} d_1 & 0 & 0 & \dots & 0 \\ 0 & d_2 & 0 & \dots & 0 \\ 0 & 0 & d_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 0 & 0 & 0 & \dots & d_N \end{bmatrix} \quad (4)$$

$$A^{-1} = (L^{-1})^T \times D^{-1} \times L^{-1} \quad (5)$$

The inversion of the lower triangular matrix is performed by the characteristics of 2×2 inverse lower triangular matrix and block matrix inversion method (6). Block matrix is considered as follows.

$$B = \begin{bmatrix} x & 0 \\ y & 1 \end{bmatrix} \quad (6)$$

Where the size of matrices is as shown.

$$x_1 \in C^{(N-1) \times (N-1)}; \text{ square matrix}$$

$$x_2 \in C^{(N-1) \times (N-2)}; \text{ copy of } x_1 \text{ matrix except last column}$$

$$y \in C^{1 \times (N-1)}; N^{\text{th}} \text{ row vector of } L \text{ matrix}$$

$$0 \in C^{(N-1) \times 1}; N^{\text{th}} \text{ zero column vector}$$

Inversion of block matrix B is obtained as (7),

$$B^{-1} = \begin{bmatrix} x_1^{-1} & 0 \\ -y \times x_2^{-1}, -l & 1 \end{bmatrix} \quad (7)$$

where l is an element immediately below diagonal elements.

From the observation we can say that if we compute X^{-1} , then it can be multiplied by one vector. It can be simply explained as the first part is utilizing those characteristics to calculate the diagonal matrix and lower triangular matrix. Further it is used to process the remaining calculation. For an instance to illustrate the process of the algorithm consider $B_{6 \times 6} \cdot B_{2 \times 2}$ and $B_{2 \times 2}^{-1}$ is obtained by the (6) and (7) respectively. Using $B_{2 \times 2}^{-1}$, (6), (7) to obtain $B_{3 \times 3}^{-1}$ till $B_{6 \times 6}^{-1}$. The algorithm is as follows.

$$L^{-1} = B_{2 \times 2}^{-1}.$$

for $n = 3$ to N

$$y = L(n, 1 : n - 1).$$

$$B_1 = L^{-1}.$$

$$B_2 = \text{zeros}(n - 1, 1).$$

$$B_3 = -[(y \times B_1(1 : n - 1, 1 : n - 2)), -L(n - 1, n - 1)].$$

$$B_4 = 1.$$

$$L^{-1} = \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix}$$

Inversion of diagonal matrix is just the inversion of diagonal elements as shown in (8).

$$D^{-1}_{N \times N} = \begin{bmatrix} 1/d_1 & 0 & 0 & \dots & 0 \\ 0 & 1/d_2 & 0 & \dots & 0 \\ 0 & 0 & 1/d_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 0 & 0 & 0 & \dots & 1/d_N \end{bmatrix} \quad (8)$$

Inversion of the matrix is obtained as follows (9).

$$A^{-1} = (L^{-1})^T \times D^{-1} \times L^{-1} \quad (9)$$

In the beginning before performing LDL^T factorization, the input matrix is converted to symmetric matrix. To compute the inverse of original matrix, multiply the A^{-1} with transposed version of original matrix as shown below.

$$\text{Original matrix inverse} = (\text{Original matrix})^T \times A^{-1}$$

Any matrix of the form lower triangular matrix or transposed upper triangular matrix with diagonal elements “1”, can be implemented using extended recursive diagonal-wise algorithm to obtain the inverse.

IV. PERFORMANCE COMPARISON

These factorization techniques are performed in MATLAB. The performance of different factorization techniques are evaluated using matrix 2-norm errors (10), considering the square matrix of dimension 1024×1024 .

$$E = \|I - HH^{-1}\|_2 \quad (10)$$

With the observations, QR has least error while the LDL^T comparably has more error, but the hardware complexity (in terms of algorithm, architecture, Look Up Tables (LUTs), memory utilization) of LDL^T is good. LDL^T has diagonal elements as "1" in L and L^T , that reduces hardware resources for computation which adds as an advantage.

Table.I shows the error comparison of various techniques computed using 500 random real matrix and complex matrix of order 1024×1024 , implemented in MATLAB with system specifications as follows – Processor : Intel core -i5 -5200U CPU @2.20GHz. Installed memory (RAM) : 8GB. Memory type : SSD. System type: 64-bit Operating System.

TABLE I
PERFORMANCE COMPARISON

Method	Norm 2 error of	
	Real Matrix	Complex Matrix
LDL^T	1.5040e-07	2.5502e-09
Cholesky	1.4608e-07	2.3021e-09
LU	1.4587e-07	2.3179e-09
QR	1.1564e-07	1.2816e-09

V. IMPLEMENTATION IN HARDWARE AND ARCHITECTURE

The proposed algorithm is implemented using Artix-7 XC7A35TICPG236C general purpose speed -1 which has 41600 FFs, 20800 LUTs, 106 IOs in Xilinx Vivado. Designed using 20ns clock cycle, signed 32bit Fixed Point notation of $Q_{6,25}$. Here 6 and 25 corresponds to integer and fraction parts respectively and 32nd bit (MSB (Most Significant Bit)) is a sign bit. For multiplication, division, vector multiplication operations in hardware, soft IP is being used. For storing input matrix, final matrix, and intermediate results distributed memory is used. Clock and reset (Asynchronous) are given as input. Inverse matrix elements are shown at the out port. The functional simulation is performed in Xilinx Vivado.

Here the top-down (hierarchical design) design approach is being followed. Datapath and Controller blocks are designed with required control signal and input signals, data signals. Input to the Datapath is a signed 32bit data in and 10bit N for defining the elements and size of a matrix ($N \times N$) respectively. Clock and reset are the two signals common for the controller and Datapath. Fig.1 shows the schematic of the elaborated design from Xilinx Vivado showing the data-path and controller with input and output signals.

Elements of the input matrix (A), Diagonal matrix (D), Lower triangular matrix (L), inverse Lower triangular matrix

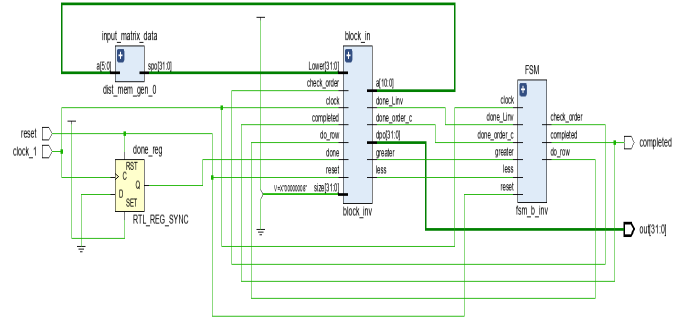


Fig. 1. Schematic of the elaborated design of extended recursive diagonal-wise algorithm (taken from Xilinx Vivado software).

(L^{-1}), inverse input matrix (A^{-1}) are mentioned in the matrix form below.

$$A = \begin{bmatrix} 2.3 & 2.9 & 4 & 2.5 \\ 2.9 & 4.5 & 5.9 & 3.4 \\ 4 & 5.9 & 8.2 & 4.7 \\ 2.5 & 3.4 & 4.7 & 3.4 \end{bmatrix} \quad (11)$$

$$D = \begin{bmatrix} 2.3 & 0 & 0 & 0 \\ 0 & 0.8435 & 0 & 0 \\ 0 & 0 & 0.3737 & 0 \\ 0 & 0 & 0 & 0.5828 \end{bmatrix} \quad (12)$$

$$L = \begin{bmatrix} 1 & 1.76e-07 & 2.39e-07 & 1.02e-07 \\ 1.2609 & 1 & 2.39e-07 & 0 \\ 1.7391 & 1.0155 & 1 & 0 \\ 1.087 & 0.2938 & 0.2690 & 1 \end{bmatrix} \quad (13)$$

$$L^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1.2609 & 1 & 0 & 0 \\ -0.4588 & -1.0155 & 1 & 0 \\ -0.5931 & -0.0207 & -0.2690 & 1 \end{bmatrix} \quad (14)$$

$$A^{-1} = \begin{bmatrix} 3.4863 & -0.2272 & -0.9538 & -1.0177 \\ -0.2272 & 3.9455 & -2.7076 & -0.0355 \\ -0.9538 & -2.7076 & 2.7999 & -0.4615 \\ -1.0177 & -0.0355 & -0.46153 & 1.7159 \end{bmatrix} \quad (15)$$

Fig. 2, Fig. 3, Fig. 4, Fig. 5, Fig. 6 show the input matrix (A) loaded in the functional simulation environment and the elements of Diagonal matrix (D), Lower triangular matrix (L), Inverse Lower triangular matrix (L^{-1}), Inverse input matrix (A^{-1}) are the outputs in the functional simulation environment.

The experiment is conducted with 8×8 size matrix (see

Input_m_1[31:0]	2.3000000	2.9000000	4.0000000	2.5000000
[0][0.3][31:0]	2.3000000	2.9000000	4.0000000	2.5000000
[1][0.3][31:0]	2.9000000	4.5000000	5.9000000	3.4000000
[2][0.3][31:0]	4.0000000	5.9000000	8.2000000	4.7000000
[3][0.3][31:0]	2.5000000	3.4000000	4.7000000	3.4000000

Fig. 2. Highlighting the input matrix (A) (equ.11) loaded in the functional simulation environment (taken from Xilinx Vivado software).

Diagonal...[31:0]	{2.30000000, {2.30000000 192093,0,0,0,0,0,0},{0,0,0.843478500843048,...
0[0:0.3][31:0]	2.30000000 192093,0,0,0,0,0,0
1[1:0.3][31:0]	0,0,0.843478500843048,0,0,0,0
2[2:0.3][31:0]	0,0,0,0,0.373711377382278,0,0
3[3:0.3][31:0]	0,0,0,0,0,0,0.582758724689484

Fig. 3. Highlighting the elements of Diagonal matrix (D) (equ.12) in the functional simulation environment (taken from Xilinx Vivado software).

Lower_fr_...[31:0]	1.089406	1.0, 0.8, 94069671630859e-08, 0.0, 0.8, 94069671630859e-08), '(1.0, 0.8, 94069671630859e-08, 0.0, 0.8, 94069671630859e-08)
0[0:3][31:0]	1.089406	1.0, 0.8, 94069671630859e-08, 0.0, 0.8, 94069671630859e-08
1[1:3][31:0]	1.2608695	1.26086953282356, 1.0, 0.5, 96046447753906e-08, 2.980232238769...
2[2:3][31:0]	1.7391304	1.73913040757179, 1.01546391844749, 1.0, 2.98023223876953e-08
3[3:3][31:0]	1.0869555	1.08695550100708, 0.293814569711685, 0.268965542316437, 1.0

Fig. 4. Highlighting the elements of Lower triangular matrix (L) (equ.13) in the functional simulation environment (taken from Xilinx Vivado software).

L_inv[0][3]:31.0]	'{1.0,0.0,0.0,0.0}','{-1.26086953282356,1.0,0.0,0.0}','{-0.458762824587628,-1.01546394824982,1.0,0.0}'
[0][0]:31.0]	1.0,0.0,0.0,0.0
[1][0]:31.0]	-1.2608695
[2][0]:31.0]	-0.4587628
[3][0]:31.0]	-0.5931033

Fig. 5. Highlighting the elements of Inverse Lower triangular matrix (L^{-1}) (equ.14) in the functional simulation environment (taken from Xilinx Vivado software).

[illegible]

Fig. 6. Highlighting the elements of Inverse input matrix (A^{-1}) (equ.15) in the functional simulation environment (taken from Xilinx Vivado software).

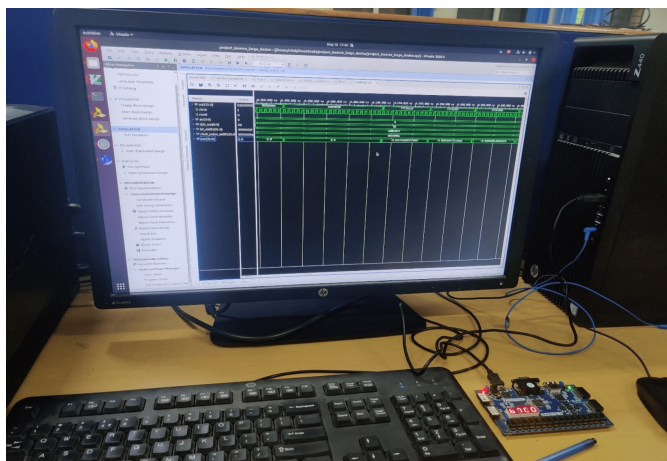


Fig. 7. Implementation setup on the Artix-7 Board

the implementation setup in Fig. 7) and the utilized resources of the five methods after the post implementation are listed in

TABLE II
UTILIZATION-POST IMPLEMENTATION OF THE EXPERIMENT

Resources	Back Substitution	SPMI	[10]	[6]	Our Algorithm
LUT	29130	16856	15867	15415	3330
FF	33380	26276	21764	18048	5658
Clock cycles	7701	6547	5094	3621	1508

Table.II and we can see that our algorithm utilizes the least resources and computes in less clock cycles.

VI. CONCLUSION

In this paper, we presented our proposed Extended Recursive Diagonal-Wise algorithm based on LDL^T decomposition. The performance of various matrix decomposition techniques was evaluated through MATLAB simulations. Implementation of proposed algorithm after LDL^T decomposition in hardware is found to be superior over other methods from the Table.II. The proposed algorithm for matrix inversion can be implemented in hardware for MIMO systems, with less resource utilization and achieve faster processing time through FPGA.

REFERENCES

- [1] F. Ito, T. Nakagawa, H. Hamazumi, and K. Fukawa, "Development of 4×4 MIMO-QPSK demodulator using complexity-reduced MLD with block QR decomposition," in 2016 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB), June 2016, pp. 1–4.
- [2] A. Burg, M. Borgmann, M. Wenk, M. Zellweger, W. Fichtner, and H. Bolcskei, "VLSI implementation of MIMO detection using the sphere decoding algorithm," *IEEE Journal of Solid-State Circuits*, vol. 40, no. 7, pp. 1566–1577, July 2005.
- [3] F. Rusek et al., "Scaling up MIMO: Opportunities and challenges with very large arrays," *IEEE Signal Process. Mag.*, vol. 30, no. 1, pp. 40–60, Jan. 2013.
- [4] A. Burian, J. Takala, and M. Ylinen, "A fixed-point implementation of matrix inversion using Cholesky decomposition," in *Circuits and Systems, 2003 IEEE 46th Midwest Symposium on*, 2003, vol. 3, pp. 1431–1434: IEEE.
- [5] M. K. Jaiswal and N. Chadrachoodan, "FPGA-based high-performance and scalable block LU decomposition architecture," *IEEE Transactions on Computers*, vol. 61, no. 1, pp. 60–72, 2011.
- [6] "Revisiting the Adjoint Matrix for FPGA Calculating the Triangular Matrix Inversion" Di Yan , Wei-Xing Wang, Lei Zuo , Member, IEEE, and Xiao-Wei Zhang. *IEEE Transactions On Circuits And Systems—II: Express Briefs*, Vol. 68, No. 6, June 2021.
- [7] An Automatic Generation and Optimization Tool for Matrix Inversion Architectures ALI IRTURK and BRIDGET BENSON University of California, San Diego SHAHNAM MIRZAEI University of California, Santa Barbara and RYAN KASTNER University of California, San Diego.
- [8] W. Xu, Y. Xi, J. Lan, and T.-F. Jiang, "An improved predictive controller on the FPGA by hardware matrix inversion," *IEEE Trans. Ind. Electron.*, vol. 65, no. 9, pp. 7395–7405, Sep. 2018.
- [9] A. Krishnamoorthy and D. Menon, "Matrix inversion using Cholesky decomposition," in *Proc. Signal Process. Algorithms Archit. Arrangements*, 2013, pp. 70–72.
- [10] X.-W. Zhang, L. Zuo, M. Li, and J.-X. Guo, "High-throughput FPGA implementation of matrix inversion for control systems," *IEEE Trans. Ind. Electron.*, early access, May 28, 2020.